

CHANDIGARH UNIVERSITY

Department of Computer Applications

PROJECT REPORT

EMPLOYEE SALARY MANAGEMENT SYSTEM

A C++ Object-Oriented Programming Project

Submitted by
ADITYA
UID: 25BCA10391
BCA – 3rd Semester

Subject: Object-Oriented Programming using C++

Submitted to: Mr. Jagvinder Singh
Department / College: BCA / UIC

CERTIFICATE

This is to certify that the project report entitled “Employee Salary Management System” is a bona fide work submitted by Aditya (UID: 25BCA10391), a student of BCA 3rd Semester, as part of the academic work for the subject Object-Oriented Programming using C++.

The project has been developed to demonstrate the practical use of classes, objects, constructors, encapsulation, member functions, calculations and formatted output in C++.

To the best of my knowledge, the work presented in this report is suitable for academic submission, subject to verification and approval by the concerned faculty member.

Faculty Signature: _____

Name of Faculty: _____

Date: _____

College Seal: _____

DECLARATION

I, Aditya, UID 25BCA10391, student of BCA 3rd Semester, hereby declare that the project entitled "Employee Salary Management System" has been prepared by me for the subject Object-Oriented Programming using C++.

The project is intended for academic learning and demonstrates the application of object-oriented programming concepts. The salary calculation rules used in the program are stated clearly in this report and may be changed according to an organization's actual salary policy.

Student Signature:

Name: Aditya

UID: 25BCA10391

Date:

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my subject teacher for providing valuable guidance, continuous support and encouragement throughout the preparation of this project entitled “Employee Salary Management System using C++.” Their explanations, suggestions and practical teaching helped me understand the important concepts of Object-Oriented Programming and apply them in a practical way. This project has given me an opportunity to improve my programming knowledge and understand how real-life problems can be solved through computer-based applications.

I am also thankful to Chandigarh University and the Department of Computer Applications for providing me with the opportunity and academic environment to complete this project as part of my BCA third-semester coursework. The knowledge gained through classroom lectures, practical sessions and study materials helped me in planning, developing and documenting this project.

I would also like to thank my classmates and friends for their valuable suggestions, discussions and cooperation during the project work. Their encouragement helped me overcome difficulties and complete different stages of the project. I am equally grateful to my family members for their patience, motivation and continuous support.

Finally, I would like to acknowledge the developers of the C++ programming language, compiler tools and online learning resources that helped me understand programming syntax, classes, objects, constructors and other concepts used in this application.

This project has been a valuable learning experience, and I sincerely thank everyone who contributed directly or indirectly to its successful completion.

ABSTRACT

The Employee Salary Management System is a menu-driven application developed using the C++ programming language to simplify the process of calculating and managing employee salary details. The main purpose of this project is to demonstrate how computer programming can be used to perform basic salary calculations accurately and systematically. The application accepts important employee information, including employee ID, employee name, designation, basic salary, allowance percentages, commission rate and number of unpaid leaves. Based on the information entered by the user, the system automatically calculates different salary components and generates a properly formatted salary slip.

The application calculates Travel Allowance (TA), Dearness Allowance (DA), House Rent Allowance (HRA) and commission according to the percentages entered by the user. It then calculates the gross salary by adding all earning components to the basic salary. After calculating the deduction for unpaid leaves, the system determines the final net salary payable to the employee. This reduces the need for repeated manual calculations and provides a consistent format for displaying salary information.

The project is developed using Object-Oriented Programming concepts in C++. A class named Employee is used to combine employee information and related functions. The application demonstrates classes, objects, encapsulation, constructors, destructors and member functions. Private data members help protect employee information, while public functions perform calculations and display salary details. A vector is also used to maintain multiple employee records during program execution.

The system provides a simple menu with options to add employees, display all salary slips, search for an employee using an ID and exit the application. It is designed as an educational project for BCA students to develop practical programming skills and understand the application of OOP concepts.

The current version stores records temporarily in memory and does not include database connectivity or permanent storage. However, it provides a foundation for future improvements such as database integration, employee login, additional salary deductions and report generation.

Keywords: C++, Object-Oriented Programming, Employee Management, Salary Calculation, Class, Object, Encapsulation, Constructor, Vector, Salary Slip.

TABLE OF CONTENTS

Chapter	Topic	Page
1	Introduction and Background	7
2	Problem Statement	8
3	Objectives and Scope	9
4	Existing and Proposed System	10
5	Requirement Analysis	11
6	OOP Concepts Used	12
7	System Design and Architecture	13
8	Algorithm and Flowchart	14
9	Salary Calculation Method	15
10	Class Design	16
11	Program Modules	17
12	Testing Strategy	18
13–17	C++ Source Code (Part 1–5)	19–23
18	Sample Output	24
19	Test Cases and Results	25
20	Advantages, Limitations and Future Scope	26
21	Conclusion and Learning Outcomes	27
22	Bibliography and Viva Questions	28

1. INTRODUCTION AND BACKGROUND

The Employee Salary Management System is a small software application developed to show how a common office activity can be handled through a C++ program. Every organization needs to calculate the salary payable to its employees for a particular month. The amount may include basic salary, travel allowance, dearness allowance, house rent allowance and commission. It may also include deductions when an employee takes unpaid leave. When these values are calculated repeatedly by hand, the process can become slow and mistakes may occur. A computer program can apply the same formula to every employee and present the result in a clear format.

This project is designed as a console-based application for learning Object-Oriented Programming. It uses an Employee class to represent an employee and to keep related information and operations together. The user enters employee details through a menu. The program creates an object, calculates the salary components and displays a salary slip. More than one employee can be handled during the same execution.

The project is intentionally kept simple so that a BCA student can understand its complete working. It uses standard C++ features such as strings, vectors, constructors, member functions, access specifiers and formatted output. It does not require a database or paid software. Records remain in memory only while the program is running. Therefore, the project is an educational model rather than a complete commercial payroll product. Its design provides a foundation for future improvements such as permanent storage, authentication and additional payroll rules.

Salary processing is a regular activity in every organization. An employee's monthly salary may include basic pay, allowances and commission. It may also include deductions such as unpaid leave. Preparing each salary slip manually can take time and may lead to calculation mistakes.

The Employee Salary Management System provides a simple way to enter employee details and calculate the salary components automatically. It displays a structured salary slip on the screen. The user can add more than one employee, view all saved slips during the current run, and search for an employee using the employee ID.

The project is implemented in C++ because the language supports classes, objects, constructors, access specifiers and reusable member functions. These features make it possible to represent an employee as an object containing both data and related behaviour.

The project is console-based. Employee records are stored in a vector in the computer's memory while the program is running. They are not permanently saved after the application is closed.

2. PROBLEM STATEMENT

The main problem addressed by this project is the difficulty of preparing consistent employee salary calculations when the work is performed manually. A person preparing salary details may need to calculate several allowances, add them to basic salary, calculate deductions and finally determine the amount payable. If there are many employees, the same steps must be repeated for every person. Repetition increases the time required and creates opportunities for arithmetic mistakes, incorrect data entry and inconsistent presentation. Manual records can also make it inconvenient to locate a particular employee's salary details. If information is written in separate registers or files, the person responsible may need to search through several entries. A spreadsheet can help, but formulas may be overwritten or copied incorrectly if the sheet is not maintained carefully. For a programming student, these manual approaches also do not demonstrate how a reusable software solution can be designed using classes and objects. The proposed application addresses these basic issues by accepting employee information once and using defined member functions to calculate each salary component. The system provides a menu to add an employee, display all stored salary slips, search by employee ID and exit. It follows the same calculation method for every record, which improves consistency. The program also checks selected invalid values, such as a non-positive employee ID or negative salary-related inputs. The project does not claim to solve every payroll problem. It does not manage taxes, provident fund, attendance databases or legal payroll.

In a small organization, salary calculations may be performed manually using a calculator or a spreadsheet. The person preparing the salary slip must calculate several components separately and then subtract deductions. Repeating these steps for multiple employees can be inconvenient.

Issues in the manual approach

- Repeated calculations are required for every employee.
- A calculation error may produce an incorrect salary amount.
- It can be difficult to maintain a consistent salary-slip format.
- Finding one employee's salary details may take extra time.
- Manual processing does not demonstrate a reusable programming solution.

Proposed solution

The proposed C++ program accepts employee details once and calculates all salary components through member functions. It uses an Employee class so that the same calculation logic can be reused for multiple objects. The menu makes the program easy to operate.

3. OBJECTIVES AND SCOPE

The primary objective of the project is to develop a menu-driven Employee Salary Management System using C++ and apply the principles of Object-Oriented Programming to a practical example. The project aims to show that employee information and the operations performed on that information can be organized inside a class. It also aims to make salary calculations repeatable, readable and easier to maintain than a collection of unrelated calculations written directly inside the main function. The specific objectives are to create an Employee class, declare suitable private data members, provide public member functions and initialize objects through constructors. The program calculates travel allowance, dearness allowance, house rent allowance and commission as percentages of basic salary. It then calculates gross salary, unpaid-leave deduction and net salary. Another objective is to display the result in a structured salary-slip format using the iomanip library. The program also demonstrates how a vector can store multiple Employee objects and how a search can be performed using an employee ID. The scope is limited to a classroom-level console application. It supports entering an employee ID, name, designation, basic salary, allowance rates, commission rate and number of unpaid leaves. Users can add records, display all available records and search for one record during the current program session. The application is suitable for demonstrating fundamental C++ programming concepts and basic testing. The current version does not permanently save records. It does not include employee login, administrator roles, database connectivity, tax calculations, provident fund, overtime,.

Objectives

1. To develop a simple employee salary application using C++.
2. To apply the concepts of class and object in a practical problem.
3. To use private and public access specifiers for encapsulation.
4. To demonstrate default and parameterized constructors.
5. To calculate TA, DA, HRA and commission from the basic salary.
6. To calculate gross salary, unpaid-leave deduction and net salary.
7. To display a clear salary slip using output manipulators.
8. To store and search multiple employee objects during one program session.

Scope

The application is intended for classroom demonstration and small sample datasets. It supports adding employee records, displaying all salary slips and searching by employee ID. The program uses user-entered rates, so different organizations can test different allowance percentages.

The current version does not include login security, database connectivity, permanent file storage, tax calculations, provident fund, overtime, payslip printing or multiple-month payroll history.

4. EXISTING AND PROPOSED SYSTEM

In an existing manual system, salary information may be maintained in registers, paper files or a spreadsheet. The person responsible collects employee details, checks the basic salary and calculates each allowance. After adding the earning components, deductions are subtracted to obtain the net amount. This method can work for a very small number of employees, but it depends heavily on careful data entry and repeated calculations. A calculation may be missed, a formula may be applied incorrectly or a record may be difficult to locate later. The format of salary details may also differ from one record to another. The proposed system is a C++ console application that performs the selected calculations through a common program. It accepts the same categories of input for each employee and uses the Employee class to organize the data. Separate member functions calculate TA, DA, HRA, commission, gross salary, leave deduction and net salary. This arrangement reduces repeated formula writing and makes the calculation process easier to understand. The menu gives the user a consistent way to add an employee, display all records, search by ID and exit. The proposed system is technically feasible because it can be compiled with a standard C++ compiler and uses commonly available language features. It is economically feasible because a free compiler and text editor can be used. It is operationally feasible for demonstration because the menu is short and the prompts guide the user through the required information. The proposed system is.

Existing manual method	Proposed C++ system
Salary components are calculated separately.	Member functions calculate each component consistently.
The same steps are repeated for every employee.	The Employee class reuses the same logic for every object.
A handwritten or manually formatted slip may vary.	The program displays a standard salary-slip layout.
Searching may require checking multiple papers.	The program searches records by employee ID.
Mistakes can occur during arithmetic.	The program performs arithmetic using defined formulas.

Feasibility

- Technical feasibility: requires a standard C++ compiler such as g++ or an IDE supporting C++17.
- Economic feasibility: uses free compiler tools and does not require paid software.
- Operational feasibility: menu options and prompts are simple for a beginner to use.

5. REQUIREMENT ANALYSIS

Requirement analysis identifies the resources and functions needed before the application is developed. The project has modest hardware requirements because it is a small console program. A desktop computer or laptop with a keyboard, monitor and sufficient memory to run an operating system and a C++ development environment is adequate. No special server, printer or network connection is required for the current version. A printer may be used separately if the displayed output needs to be captured for academic submission. The software requirement is an operating system such as Windows, Linux or macOS and a C++ compiler that supports the language features used in the source code. The program is suitable for a C++11 or later compiler and is intended to be compiled using C++17. An editor or integrated development environment such as Visual Studio Code, Code::Blocks, Dev-C++ or Visual Studio can be used to write and build the source file. The standard headers used by the program include iostream, iomanip, vector, string and limits. The functional requirements describe what the system must do. It must accept employee ID, employee name, designation, basic salary, allowance rates, commission rate and unpaid-leave count. It must calculate the salary components, display a salary slip, display all records held in the current session and search for a record using its ID. It must also provide an option to exit. The non-functional requirements describe the expected quality of operation. The interface should be understandable, calculations should use consistent formulas, and.

Hardware requirements

- Desktop computer or laptop.
- Minimum 2 GB RAM recommended for a basic C++ development environment.
- Keyboard and monitor.

Software requirements

- Operating system: Windows, Linux or macOS.
- C++ compiler supporting C++11 or later; the source is compatible with C++17.
- Editor or IDE: Visual Studio Code, Code::Blocks, Dev-C++, Visual Studio or any text editor with a compiler.

Functional requirements

9. The system shall accept employee ID, name and designation.
10. The system shall accept basic salary, TA, DA, HRA and commission rates.
11. The system shall accept the number of unpaid leaves.
12. The system shall calculate and display the salary slip.
13. The system shall display all employee slips in the current session.
14. The system shall search for an employee by ID.
15. The system shall allow the user to exit from the menu.

Non-functional requirements

- Usability: clear menu and readable output.
- Reliability: calculations follow the same formulas for each employee.
- Maintainability: salary calculations are separated into member functions.
- Portability: standard C++ features are used.

6. OOP CONCEPTS USED

Object-Oriented Programming is a programming approach in which a program is organized around objects that combine data and the functions that operate on that data. This project uses the Employee class as its main design element. The class describes the information associated with an employee and provides operations for calculating and displaying salary details. Each employee entered by the user is represented by an object created from this class. Encapsulation is demonstrated by declaring the employee data members under the private access specifier. These members include the employee ID, name, designation, basic salary, allowance rates, leave count and commission rate. The main function does not directly modify these private values. Instead, the public constructors initialize the values and public member functions perform the required operations. This keeps the data and related behaviour together and limits direct access to internal details. Constructors are used to initialize objects. The default constructor assigns safe initial values such as zero and “Not Set”. The parameterized constructor receives the actual values entered by the user. Member functions such as calculateTA(), calculateDA(), calculateHRA(), calculateGrossSalary() and calculateNetSalary() divide the work into smaller operations. This makes the program easier to read and reduces repeated calculations. The class also contains a destructor, which demonstrates the syntax and role of object cleanup. The program does not allocate memory manually, so the destructor does not need to release custom resources. A vector<Employee> stores a collection of objects during execution. The use of these concepts makes the.

Class and object

Employee is the class. It defines the data and operations common to an employee. Each employee entered by the user becomes an object of this class.

Encapsulation

Employee details are declared under private access. Public functions calculate salary values and display the salary slip. This prevents main() from directly changing the private data members.

Constructors

The default constructor initializes an Employee object with safe initial values. The parameterized constructor receives actual employee details and initializes a complete employee object.

Member functions

Functions such as calculateTA(), calculateDA(), calculateHRA(), calculateGrossSalary(), calculateNetSalary() and displaySalarySlip() define the behaviour of an Employee object.

Object collection

A vector<Employee> stores multiple objects. The vector grows as new employees are added. The range-based for loop is used to display or search these objects.

Manipulators

The program uses setw(), fixed, setprecision(), and left to format the salary slip. These are provided by the iomanip header.

7. SYSTEM DESIGN AND ARCHITECTURE

The system follows a simple layered flow even though it is implemented in a single C++ source file. The user interaction layer is controlled by the main() function. It displays the menu, accepts the user's choice and reads the information required for each operation. The data and calculation layer is represented by the Employee class. This class stores the employee details and provides functions that calculate salary components. The output layer is the salary-slip display function, which presents the employee details and calculated amounts in a readable format. When the program starts, main() creates an empty vector of Employee objects. The menu is displayed repeatedly until the user chooses the exit option. If the user selects the add-employee option, the program reads the employee details and creates an Employee object by calling the parameterized constructor. The object is added to the vector. The program then calls displaySalarySlip() to show the result. If the user selects display-all, a loop visits each object in the vector and displays its salary slip. If search is selected, the program compares the entered ID with the ID returned by each object's getId() function. The architecture keeps the calculation logic inside the class rather than placing every formula in main(). This separation improves readability and makes it easier to change a formula in one place. The program performs all processing locally. It does not communicate with a remote server or database, and its vector is cleared when the program ends. This.

The application follows a simple console-based design. The main() function controls the menu and user interaction. The Employee class contains employee data and salary calculation logic.

High-level architecture

Layer / component	Responsibility
User interface (main function)	Shows menu, accepts input and selects an operation.
Employee class	Stores employee information in private data members.
Calculation functions	Calculate allowances, commission, gross salary, deduction and net salary.
Display function	Prints a formatted salary slip.
Vector collection	Keeps Employee objects available during the current program session.

Data flow

User input → Employee object creation → Salary component calculations → Gross salary and leave deduction → Net salary → Salary slip displayed on screen.

The application does not connect to a server or database. All processing takes place locally in the running program.

8. ALGORITHM AND FLOWCHART

An algorithm is a finite sequence of clear steps used to solve a problem. The algorithm of this project begins by starting the program and creating an empty vector to hold Employee objects. The system then displays the main menu and asks the user to enter a choice. The menu is repeated until the user selects the exit option, which makes the application convenient for entering or viewing more than one employee record in a single session. When the user selects option one, the program asks for the employee ID, name, designation, basic salary, allowance percentages, number of unpaid leaves and commission percentage. It checks selected values and, if the values are acceptable, creates an Employee object using the parameterized constructor. The object is added to the vector. The salary-slip function is then called to calculate and display the employee's salary details. When option two is selected, the program checks whether the vector is empty. If there are no records, it displays an appropriate message. Otherwise, it uses a loop to display every employee's salary slip. When option three is selected, the program asks for an employee ID and compares it with the ID of each object. If a match is found, the corresponding salary slip is displayed; otherwise, a not-found message appears. Option four displays a closing message and terminates the loop.

Algorithm

16. Start the program.
17. Create an empty vector to store Employee objects.
18. Display the main menu.
19. Read the user's choice.
20. If choice is 1, read employee details and validate basic numeric values.
21. Create an Employee object and add it to the vector.
22. Calculate salary components and display the employee's salary slip.
23. If choice is 2, display all stored salary slips, or show a message if there are no records.
24. If choice is 3, ask for an employee ID and search the vector.
25. If choice is 4, display the exit message and stop.
26. For any other choice, display an invalid-choice message.
27. Repeat the menu until the user selects Exit.

Text flowchart

START
↓
Display Menu
↓
Read Choice
↓
Choice = 1? → Enter details → Create object → Calculate salary → Display slip
↓ No
Choice = 2? → Display all employee slips
↓ No
Choice = 3? → Search employee ID → Display result
↓ No
Choice = 4? → Exit
↓ Otherwise
Show invalid choice

↓
Return to Menu

9. SALARY CALCULATION METHOD

The salary calculation method used in this project is based on percentages entered by the user. Basic salary is treated as the starting amount. Travel Allowance (TA), Dearness Allowance (DA), House Rent Allowance (HRA) and commission are each calculated as a percentage of basic salary. This approach makes the demonstration flexible because the user can enter different rates for different sample employees. The rates are not fixed by the program and should be understood as sample inputs rather than official payroll rules. The formula for an allowance is: allowance amount = basic salary multiplied by the allowance rate divided by 100. The same method is used for TA, DA, HRA and commission. Gross salary is calculated by adding basic salary and all four earning components. The project then calculates an unpaid-leave deduction. For this academic model, one month is assumed to contain 30 days. The daily basic salary is therefore basic salary divided by 30. The deduction is daily basic salary multiplied by the number of unpaid leaves. Net salary is calculated by subtracting the leave deduction from gross salary. For example, when basic salary is Rs. 30,000, TA is 10%, DA is 20%, HRA is 15%, commission is 5% and unpaid leaves are two, the gross salary becomes Rs. 45,000. The leave deduction becomes Rs. 2,000, giving a net salary of Rs. 43,000. The model does not include tax, provident fund, insurance, overtime or other deductions. The 30-day month and percentage-based rates are assumptions.

The program uses percentage-based allowances. The rates are entered by the user as percentages of basic salary. This makes the program flexible for demonstration.

Formulas

Component	Formula
Travel Allowance (TA)	Basic Salary × TA Rate ÷ 100
Dearness Allowance (DA)	Basic Salary × DA Rate ÷ 100
House Rent Allowance (HRA)	Basic Salary × HRA Rate ÷ 100
Commission	Basic Salary × Commission Rate ÷ 100
Gross Salary	Basic + TA + DA + HRA + Commission
Leave Deduction	(Basic Salary ÷ 30) × Unpaid Leaves
Net Salary	Gross Salary − Leave Deduction

Important assumptions

- Each allowance and commission rate is entered as a percentage of basic salary.
- One month is treated as 30 days for leave-deduction calculation.
- Every leave entered is treated as an unpaid leave.
- No income tax, PF, insurance or other deductions are included.
- These are academic assumptions, not a statement of any legal or employer payroll policy.

10. CLASS DESIGN

The central class in the application is named Employee. It represents one employee and contains the information required to prepare that employee's salary slip. The class is designed so that the data and operations related to an employee remain together. This is an important feature of object-oriented design because it makes the program easier to understand and extend.

The private section contains data members for employeeId, employeeName, designation, basicSalary, taRate, daRate, hraRate, leavesTaken and commissionRate. These variables store the values entered by the user. They are private so that other parts of the program cannot access or change them directly. This supports encapsulation and reduces accidental modification of employee data.

The public section contains two constructors. The default constructor initializes the members to safe starting values. The parameterized constructor accepts the employee information and assigns it to the corresponding data members. The class also contains calculation functions. calculateTA(), calculateDA(), calculateHRA() and calculateCommission() return the value of each earning component. calculateGrossSalary() adds the basic salary and allowances. calculateLeaveDeduction() calculates the deduction based on unpaid leaves. calculateNetSalary() returns gross salary minus the deduction.

The displaySalarySlip() function prints the employee information and calculated amounts using formatted output. The getId() function returns the employee ID so that main() can search for an employee without directly accessing the private variable. A destructor is included to demonstrate object destruction. Since the class uses standard library objects and does not manually allocate memory, no custom cleanup is required. This class design keeps the program modular and readable.

Class: Employee

Type	Member	Purpose
Private data	employeeId	Unique employee identification number.
Private data	employeeName	Stores employee name.
Private data	designation	Stores job title.
Private data	basicSalary	Stores monthly basic salary.
Private data	taRate, daRate, hraRate	Stores allowance rates.
Private data	leavesTaken	Stores unpaid leave count.
Private data	commissionRate	Stores commission percentage.
Public function	Employee()	Default constructor.
Public function	Employee(...)	Parameterized constructor.
Public functions	calculateTA/DA/HRA()	Calculate individual allowances.
Public functions	calculateGrossSalary()	Calculate total earnings.
Public functions	calculateNetSalary()	Calculate final salary after leave deduction.
Public function	displaySalarySlip()	Prints salary details.
Public function	getId()	Returns employee ID for searching.
Public function	~Employee()	Destructor.

11. PROGRAM MODULES

The program is divided logically into modules so that each part performs a specific responsibility. Although the source code is written in one file, the functions and menu branches separate the work into understandable units. The first module is employee initialization. The default constructor initializes an object with safe values, while the parameterized constructor receives the details entered by the user. This ensures that an object has its data assigned when it is created. The second module is salary calculation. Separate member functions calculate TA, DA, HRA and commission. Each function applies the percentage formula to the basic salary. The gross salary function adds all earning components. The leave deduction function calculates the amount to be deducted for unpaid leave, and the net salary function subtracts this deduction from gross salary. Keeping calculations separate makes it easier to check and maintain the formulas. The third module is salary-slip display. The displaySalarySlip() function prints the employee ID, name, designation, basic salary, allowances, commission, gross salary, leave count, leave deduction and net salary. The iomanip library is used to align labels and display monetary amounts with two digits after the decimal point. The fourth module is record and menu management. The main() function displays the menu and uses a vector to store Employee objects. It supports adding a record, displaying all records, searching by ID and exiting. The fifth module is input checking. It rejects selected invalid values, such as a non-positive ID or negative salary and.

Module 1: Employee initialization

The constructors initialize the employee's data. The default constructor sets values to zero or “Not Set”. The parameterized constructor accepts the employee details supplied by the user.

Module 2: Salary calculations

Separate functions calculate TA, DA, HRA and commission. Gross salary adds basic salary and all earnings. Leave deduction is calculated separately and subtracted to obtain net salary.

Module 3: Salary slip display

displaySalarySlip() prints the employee details and each salary component. The iomanip manipulators align the output and display amounts to two decimal places.

Module 4: Menu and records

main() displays four menu options. A vector stores the objects. The user can add an employee, display all salary slips, search by ID or exit.

Module 5: Input checks

The program checks whether menu input is numeric and rejects a non-positive employee ID or negative salary, rate or leave count. A full production payroll system would require additional validation.

12. TESTING STRATEGY

Testing is the process of checking whether a program behaves as expected for selected inputs. The Employee Salary Management System should be tested with normal values, boundary values and invalid entries. The purpose is to verify both the arithmetic calculations and the menu operations. Testing should be performed after compiling the source code so that syntax or compilation errors can be identified before the program is evaluated. A normal test begins by adding an employee with a positive ID, a name, a designation, a basic salary of Rs. 30,000, TA rate of 10%, DA rate of 20%, HRA rate of 15%, commission rate of 5% and two unpaid leaves. The expected TA is Rs. 3,000, DA is Rs. 6,000, HRA is Rs. 4,500 and commission is Rs. 1,500. The expected gross salary is Rs. 45,000. The daily basic salary under the 30-day assumption is Rs. 1,000, so two unpaid leaves produce a deduction of Rs. 2,000. The expected net salary is Rs. 43,000. The display-all option should be tested after adding more than one employee. The search option should be tested with both an existing ID and a non-existing ID. The empty-record condition should be tested by selecting display-all before adding any employee. Invalid menu choices and negative numeric values should also be entered to observe the program's response. Testing results should be compared with the expected calculations. If a result differs, the input, formula and.

Testing is performed by entering sample employee information and checking whether the displayed values follow the stated formulas. The test data below is illustrative.

Input item	Sample value
Employee ID	101
Employee name	Aarav Sharma
Designation	Assistant Manager
Basic salary	Rs. 30,000.00
TA rate	10%
DA rate	20%
HRA rate	15%
Unpaid leaves	2
Commission rate	5%

Expected calculations

- TA = 30,000 × 10 / 100 = Rs. 3,000.00
- DA = 30,000 × 20 / 100 = Rs. 6,000.00
- HRA = 30,000 × 15 / 100 = Rs. 4,500.00
- Commission = 30,000 × 5 / 100 = Rs. 1,500.00
- Gross = 30,000 + 3,000 + 6,000 + 4,500 + 1,500 = Rs. 45,000.00
- Leave deduction = (30,000 / 30) × 2 = Rs. 2,000.00
- Net salary = 45,000 − 2,000 = Rs. 43,000.00

13. C++ SOURCE CODE – PART 1

Part 1 of the source code introduces the libraries and declares the Employee class. The program includes iostream for input and output operations, iomanip for formatting the salary slip, vector for storing multiple employee objects, string for employee names and designations, and limits for clearing invalid input from the input stream. The using namespace std statement allows standard library names to be used without writing the std:: prefix each time. The class declaration begins with the class keyword followed by the name Employee. Its private section lists the data members that describe an employee. These include the ID, name, designation, basic salary, allowance rates, leave count and commission rate. Declaring these members as private demonstrates encapsulation. The data is not directly available to the main function; it is managed through the class's public interface. The public section begins with the default constructor. A constructor is a special member function that is called automatically when an object is created. The default constructor assigns initial values so that the object does not contain uninitialized data. The parameterized constructor accepts the values supplied by the user and assigns them to the object's data members. This allows the program to create a fully initialized Employee object in one statement. This first part establishes the structure used by the rest of the program. The class declaration is important because the calculation and display functions are written as members of this class. When studying this part, focus on the difference between.

The following is the complete program. It is written in standard C++ and uses the Employee class. The code is divided across report pages for readability. A separate .cpp file is also provided with this report.

Header files and class declaration

```
001  #include <iostream>
002  #include <iomanip>
003  #include <vector>
004  #include <string>
005  #include <limits>
006  using namespace std;
007
008  // Employee class demonstrates encapsulation, constructors and member functions.
009  class Employee {
010  private:
011      int employeeId;
012      string employeeName;
013      string designation;
014      double basicSalary;
015      double taRate;
016      double daRate;
017      double hraRate;
018      int leavesTaken;
019      double commissionRate;
020
021  public:
022      // Default constructor
023      Employee() {
024          employeeId = 0;
025          employeeName = "Not Set";
026          designation = "Not Set";
027          basicSalary = 0.0;
028          taRate = 0.0;
029          daRate = 0.0;
030          hraRate = 0.0;
031          leavesTaken = 0;
032          commissionRate = 0.0;
033      }
034
035      // Parameterized constructor
036      Employee(int id, string name, string post, double basic,
037               double ta, double da, double hra, int leaves, double commission) {
038          employeeId = id;
039          employeeName = name;
040          designation = post;
```

```
041         basicSalary = basic;
042         taRate = ta;
043         daRate = da;
044         hraRate = hra;
045         leavesTaken = leaves;
046         commissionRate = commission;
047     }
048
```

14. C++ SOURCE CODE – PART 2

Part 2 contains the member functions that perform the salary calculations and display the salary slip. The functions calculateTA(), calculateDA(), calculateHRA() and calculateCommission() each return a double value. They multiply the basic salary by the relevant rate and divide the result by 100. Returning a value allows the result to be reused by other member functions without storing unnecessary duplicate values.

The calculateLeaveDeduction() function uses the assumption that one month contains 30 days. It divides the basic salary by 30 to obtain the daily basic amount and multiplies that amount by the number of unpaid leaves. The calculateGrossSalary() function adds the basic salary to TA, DA, HRA and commission. The calculateNetSalary() function subtracts the leave deduction from gross salary. These functions are declared const because they calculate values without changing the employee's stored data.

The displaySalarySlip() function presents the employee information and all calculated amounts. It uses cout to print labels and values. The setw() manipulator reserves a fixed width for a field, left aligns the text, fixed selects fixed-point notation and setprecision(2) displays two digits after the decimal point. These formatting tools make the output easier to read.

The getId() function returns the employee ID and allows the main function to search for a record while keeping the data member private. Part 2 demonstrates how a class can provide a clear public interface for calculations and display operations. It also shows why small member functions are easier to test and maintain than one large block of code.

```
049     // Member function to calculate salary components
050     double calculateTA() const { return basicSalary * taRate / 100.0; }
051     double calculateDA() const { return basicSalary * daRate / 100.0; }
052     double calculateHRA() const { return basicSalary * hraRate / 100.0; }
053     double calculateCommission() const { return basicSalary * commissionRate / 100.0; }
054
055     // Assumption: each unpaid leave deducts 1/30th of basic salary.
056     double calculateLeaveDeduction() const {
057         return (basicSalary / 30.0) * leavesTaken;
058     }
059
060     double calculateGrossSalary() const {
061         return basicSalary + calculateTA() + calculateDA()
062                + calculateHRA() + calculateCommission();
063     }
064
065     double calculateNetSalary() const {
066         return calculateGrossSalary() - calculateLeaveDeduction();
067     }
068
069     void displaySalarySlip() const {
070         cout << "\n==================================================\n";
071         cout << "                 EMPLOYEE SALARY SLIP\n";
072         cout << "==================================================\n";
073         cout << left << setw(25) << "Employee ID:" << employeeId << '\n';
074         cout << left << setw(25) << "Employee Name:" << employeeName << '\n';
075         cout << left << setw(25) << "Designation:" << designation << '\n';
076         cout << "--------------------------------------------------\n";
```

```
        cout << fixed << setprecision(2);
        cout << left << setw(35) << "Basic Salary (Rs.):" << basicSalary << '\n';
        cout << left << setw(35) << "Travel Allowance - TA (Rs.):" << calculateTA() << '\n';
        cout << left << setw(35) << "Dearness Allowance - DA (Rs.):" << calculateDA() << '\n';
        cout << left << setw(35) << "House Rent Allowance - HRA (Rs.):" << calculateHRA() << '\n';
        cout << left << setw(35) << "Commission (Rs.):" << calculateCommission() << '\n';
        cout << "-------------------------------------------------\n";
        cout << left << setw(35) << "Gross Salary (Rs.):" << calculateGrossSalary() << '\n';
        cout << left << setw(35) << "Leaves Taken:" << leavesTaken << '\n';
        cout << left << setw(35) << "Leave Deduction (Rs.):" << calculateLeaveDeduction() << '\n';
        cout << "-------------------------------------------------\n";
        cout << left << setw(35) << "NET SALARY (Rs.):" << calculateNetSalary() << '\n';
        cout << "=================================================\n";
    }

    int getId() const { return employeeId; }

    // Destructor
    ~Employee() {
        // No manually allocated memory is used in this class.
```

14. C++ SOURCE CODE – PART 3

Part 3 completes the Employee class and begins the main() function. The destructor is written using the class name preceded by the tilde symbol. A destructor is called automatically when an object is destroyed. In this project, the destructor does not perform a special cleanup operation because the class does not allocate memory manually. It is included to demonstrate the syntax and basic purpose of a destructor in C++. After the class definition, the main() function becomes the starting point of program execution. A vector named employees is declared to store Employee objects. A vector is useful because the number of employees is not fixed in advance. New objects can be added as the user enters records. The variable choice stores the menu selection made by the user. The do-while loop is used to display the menu at least once and repeat it until the user selects option four. The menu contains four choices: add an employee and generate a salary slip, display all salary slips, search for an employee by ID and exit. The program reads the selected choice. If the input is not numeric, the stream state is cleared and the remaining invalid characters are ignored. The loop then returns to the menu instead of continuing with an unusable value. This part demonstrates the use of a vector, a loop and basic input handling. It connects the Employee class to the user interface. When reviewing the code, notice that main() controls the sequence.

```
    }
};

int main() {
    vector<Employee> employees;
    int choice;

    do {
        cout << "\n========== EMPLOYEE SALARY MANAGEMENT SYSTEM ==========\n";
        cout << "1. Add employee and generate salary slip\n";
        cout << "2. Display all salary slips\n";
        cout << "3. Search employee by ID\n";
        cout << "4. Exit\n";
        cout << "Enter your choice: ";

        if (!(cin >> choice)) {
            cout << "Invalid input. Please enter a number.\n";
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            continue;
        }

        if (choice == 1) {
            int id, leaves;
            string name, post;
```

```
            double basic, ta, da, hra, commission;

            cout << "Enter employee ID: ";
            cin >> id;
            cin.ignore(numeric_limits<streamsize>::max(), '\n');

            cout << "Enter employee name: ";
            getline(cin, name);

            cout << "Enter designation: ";
            getline(cin, post);

            cout << "Enter basic salary (Rs.): ";
            cin >> basic;
            cout << "Enter TA rate (% of basic salary): ";
            cin >> ta;
            cout << "Enter DA rate (% of basic salary): ";
            cin >> da;
            cout << "Enter HRA rate (% of basic salary): ";
            cin >> hra;
            cout << "Enter number of unpaid leaves taken: ";
            cin >> leaves;
            cout << "Enter commission rate (% of basic salary): ";
```

14. C++ SOURCE CODE – PART 4

Part 4 contains the main menu operations for adding, displaying and searching employee records. When the user selects option one, local variables are declared to receive the employee ID, leave count, name, designation, basic salary, allowance rates and commission rate. The program reads the ID and then uses getline() for the employee name and designation. getline() is useful because names and designations may contain spaces.

After collecting the values, the program checks whether the ID is positive and whether the salary, rates and leave count are non-negative. If the values fail these checks, an error message is displayed. Otherwise, employees.emplace_back() constructs a new Employee object at the end of the vector using the entered values. The program confirms that the employee was added and displays the salary slip of the newly created object.

For option two, the program checks whether the vector is empty. If it contains no records, a message is displayed. If records are available, a range-based for loop visits each Employee object and calls displaySalarySlip(). For option three, the program reads the ID to search for and checks each object by calling getId(). When a match is found, the salary slip is displayed and the search stops. If no match is found, the program displays a not-found message.

Option four displays a closing message. Any other menu selection produces an invalid-choice message. These branches demonstrate if, else-if, loops, object methods and vector operations in a practical program.

```
            cin >> commission;

            if (id <= 0 || basic < 0 || ta < 0 || da < 0 || hra < 0 ||
                leaves < 0 || commission < 0) {
                cout << "Invalid values. Please enter positive ID and non-negative amounts/rates.\n";
            } else {
                employees.emplace_back(id, name, post, basic, ta, da, hra, leaves, commission);
                cout << "\nEmployee added successfully.\n";
                employees.back().displaySalarySlip();
            }
        } else if (choice == 2) {
            if (employees.empty()) {
                cout << "No employee records available.\n";
            } else {
                for (const Employee &emp : employees) {
                    emp.displaySalarySlip();
```

```
                }
            }
        } else if (choice == 3) {
            int searchId;
            bool found = false;
            cout << "Enter employee ID to search: ";
            cin >> searchId;

            for (const Employee &emp : employees) {
                if (emp.getId() == searchId) {
                    emp.displaySalarySlip();
                    found = true;
                    break;
                }
            }
            if (!found) {
                cout << "Employee record not found.\n";
            }
        } else if (choice == 4) {
            cout << "Thank you for using Employee Salary Management System.\n";
        } else {
            cout << "Invalid choice. Please select 1 to 4.\n";
        }
    } while (choice != 4);

    return 0;
}
```

18. SAMPLE OUTPUT

The sample output section shows how a user interacts with the program after it has been compiled and executed. The application first displays the Employee Salary Management System menu. The user chooses option one to add an employee. The program then asks for the employee ID, name, designation, basic salary, allowance rates, unpaid leaves and commission rate. The sample record uses employee ID 101, employee name Aarav Sharma and designation Assistant Manager. The basic salary is entered as Rs. 30,000. The user enters TA as 10%, DA as 20%, HRA as 15%, two unpaid leaves and commission as 5%. After the values are accepted, the program displays a confirmation message and prints the salary slip. The slip shows each earning component separately so that the user can understand how the final amount is calculated. TA is Rs. 3,000, DA is Rs. 6,000, HRA is Rs. 4,500 and commission is Rs. 1,500. These amounts are added to basic salary to produce a gross salary of Rs. 45,000. The leave deduction is calculated using the 30-day month assumption. The daily basic amount is Rs. 1,000, and two unpaid leaves produce a deduction of Rs. 2,000. The resulting net salary is Rs. 43,000. Monetary values are displayed with two decimal places to make the output look like a salary statement. The sample output is illustrative. It demonstrates one successful employee entry and the expected calculation. Other employee details will.

The following sample is based on the test data in Chapter 12. The employee enters the information through the menu. The program then displays the salary slip.

```
========== EMPLOYEE SALARY MANAGEMENT SYSTEM ==========
1. Add employee and generate salary slip
2. Display all salary slips
3. Search employee by ID
4. Exit
Enter your choice: 1
Enter employee ID: 101
Enter employee name: Aarav Sharma
Enter designation: Assistant Manager
Enter basic salary (Rs.): 30000
Enter TA rate (% of basic salary): 10
Enter DA rate (% of basic salary): 20
Enter HRA rate (% of basic salary): 15
Enter number of unpaid leaves taken: 2
Enter commission rate (% of basic salary): 5


Employee added successfully.
                EMPLOYEE SALARY SLIP
Employee ID:                    101
Employee Name:                  Aarav Sharma
Designation:                    Assistant Manager
Basic Salary (Rs.):             30000.00
Travel Allowance - TA (Rs.):    3000.00
Dearness Allowance - DA (Rs.):  6000.00
House Rent Allowance - HRA:     4500.00
Commission (Rs.):               1500.00
Gross Salary (Rs.):             45000.00
Leaves Taken:                   2
Leave Deduction (Rs.):          2000.00
NET SALARY (Rs.):               43000.00
```

19. TEST CASES AND RESULTS

Test cases are prepared to check whether the application produces the expected response for different conditions. Each test case contains an input situation, the expected behaviour and the observed result. A normal employee-entry test checks whether valid employee details are accepted and a salary slip is generated. Using the sample values from the calculation chapter, the expected net salary is Rs. 43,000. The displayed result should match this value when the same inputs are entered. A second test checks the display-all option after two or more employees have been added. The expected result is that the program displays a salary slip for each object stored in the vector. A third test checks the search feature using an employee ID that exists in the vector. The expected result is the matching employee's salary slip. A fourth test uses an ID that is not present. The expected result is the message that the employee record was not found. An empty-list test selects display-all before adding any employee. The expected result is a message that no employee records are available. An invalid-menu test enters a number outside the range one to four. The program should display an invalid-choice message and return to the menu. A negative-value test enters a negative salary, rate or leave count. The current validation should reject these values. The source code was compiled successfully in the development environment used to prepare this project. The tests described here are suitable for repeating during a project.

Test ID	Test condition	Expected result
TC-01	Add employee with valid details	Employee is added and salary slip is displayed.
TC-02	Display all when records exist	All stored salary slips are displayed.
TC-03	Display all when no record exists	Message: No employee records available.
TC-04	Search using existing ID	Matching employee salary slip is displayed.
TC-05	Search using unknown ID	Message: Employee record not found.
TC-06	Enter menu option outside 1–4	Message: Invalid choice.
TC-07	Enter negative basic salary or rate	Employee record is rejected.
TC-08	Enter a non-numeric menu choice	Input error message is displayed and menu repeats.

Result

The test cases describe the expected behaviour of the program. The representative build and run can be checked using the separate C++ source file. Actual results may vary slightly in spacing depending on the terminal.

Compilation successful.

20. ADVANTAGES, LIMITATIONS AND FUTURE SCOPE

The Employee Salary Management System has several advantages as a beginner-level programming project. It provides a simple menu that guides the user through the available operations. Salary calculations are performed using separate member functions, which helps keep the logic organized and makes the formulas easier to review. The Employee class demonstrates how data and related functions can be grouped together. A vector allows the program to handle multiple employee objects without deciding the number of records in advance. The formatted salary slip also makes the output clear and suitable for a classroom demonstration. The project has important limitations. Employee records are stored only in the computer's memory while the application is running. When the program closes, the records are lost. The system does not provide a login facility or different access permissions for employees and administrators. It calculates only the earning components and unpaid-leave deduction defined in the project. It does not calculate income tax, provident fund, insurance, overtime, bonus or other organization-specific amounts. Input validation is limited, and the program does not prevent duplicate employee IDs. It also does not connect to a database or generate printable PDF payslips. The future scope of the project includes adding file handling or a database so records can be saved permanently. Update and delete options could be added to manage employee information. A login system could provide controlled access. More payroll components could be included according to an approved policy. The application could also generate monthly reports,.

Advantages

- Easy to understand and operate through a menu.
- Demonstrates important OOP concepts in one practical example.
- Calculates salary components consistently.
- Supports multiple employee objects in one session.
- Uses separate member functions, making the code easier to maintain.

Limitations

- Records are stored only in memory and disappear when the program closes.
- No password or role-based access is provided.
- Only the specified allowances and leave deduction are calculated.
- The program does not connect to a database or generate a PDF payslip.
- Input validation is basic and can be improved.

Future enhancements

- Add file handling or a database to store records permanently.
- Add employee update and delete options.
- Add tax, PF, bonus, overtime and other deductions according to an approved policy.
- Add a login system and access permissions.
- Generate printable PDF salary slips and monthly reports.
- Create a graphical user interface.

21. CONCLUSION AND LEARNING OUTCOMES

The Employee Salary Management System successfully demonstrates how a basic salary calculation task can be represented as a C++ object-oriented programming project. The application accepts employee details, creates objects from the Employee class and calculates the earning components and deductions using member functions. It displays the employee information and final net salary in a structured salary slip. The menu also allows the user to add multiple employees, display all available records and search for an employee using an ID during one program session. The main learning outcome is an understanding of how a class can represent a real-world entity. The Employee class combines data members with the functions that operate on them. The project also demonstrates encapsulation by keeping data members private and providing public functions for controlled access. Constructors are used to initialize objects, and the destructor introduces the object-cleanup concept. Separate member functions show how a larger task can be divided into smaller operations. The project provides practice with standard C++ input and output, strings, vectors, loops, conditional statements and formatted printing. It also gives experience in compiling a source file, entering test data, checking expected calculations and documenting the design. These skills are useful for later programming subjects and larger software projects. The current application is an educational prototype and should not be treated as a complete payroll solution for an organization. Permanent storage, stronger validation, security and additional payroll rules can be implemented in future versions. Overall, the project meets its.

The Employee Salary Management System demonstrates how a real-life salary calculation task can be implemented using C++ and object-oriented programming. The Employee class stores employee details and provides functions to calculate allowances, commission, gross salary, leave deduction and net salary.

Through this project, the concepts of classes, objects, private and public access, constructors, destructors, member functions, vectors and output manipulators are applied in a single program. The menu-driven design makes it possible to add, display and search employee records during one execution.

Learning outcomes

- Understanding how to design a class for a real-world entity.
- Creating objects using constructors.
- Applying encapsulation through access specifiers.
- Writing reusable member functions for calculations.
- Using a vector to manage a collection of objects.
- Formatting console output with iomanip.
- Compiling, testing and documenting a C++ program.

Overall, the project fulfils its academic objective as a beginner-friendly OOP application. Permanent storage and advanced payroll features can be added in future versions.

22. BIBLIOGRAPHY AND VIVA QUESTIONS

The bibliography lists the learning materials and references that support the development and understanding of this project. A programming project should identify the books, documentation and course materials consulted during its preparation. The references below provide explanations of C++ syntax, object-oriented programming, standard library features and the academic concepts used in the Employee Salary Management System. Students should follow the citation format prescribed by their department if a specific format is required. Bjarne Stroustrup's The C++ Programming Language is a detailed reference for the C++ language and its design. Herbert Schildt's C++: The Complete Reference provides explanations and examples of core language features. The cppreference website is a useful online reference for language syntax and standard library components such as vector, string, iostream and iomanip. The subject syllabus and classroom notes are relevant for matching the project with the topics taught in the Object-Oriented Programming using C++ course. Viva questions help a student explain the project rather than only submit the printed report. A student should be prepared to define a class and object, explain encapsulation, identify private and public members, and describe the purpose of constructors and destructors. The student should also explain why a vector is used, how the allowance percentages are applied, and how gross salary differs from net salary. It is important to explain the 30-day month assumption used for unpaid leave deduction. During a demonstration, the student can compile the program, add a sample employee, display the salary slip and.

References

- Bjarne Stroustrup, The C++ Programming Language, Addison-Wesley.
- Herbert Schildt, C++: The Complete Reference, McGraw-Hill.
- cppreference.com – C++ language and standard library reference: https://en.cppreference.com/
- Course syllabus: Object-Oriented Programming using C++, Unit 1 – Introduction to OOP and Experiment 1.2 (salary slip using classes and objects).

Possible viva questions

1. What is a class? A class is a user-defined type that combines data members and member functions.
2. What is an object? An object is an instance of a class.
3. What is encapsulation? Encapsulation combines data and functions and controls access to internal data.
4. Why are data members private? To prevent direct access and support-controlled use through public functions.
5. What is a constructor? A special member function called automatically when an object is created.
6. What is a parameterized constructor? A constructor that receives values as parameters.
7. What is a destructor? A special member function called when an object is destroyed.
8. What is TA? Travel Allowance; in this project it is calculated as a percentage of basic salary.
9. How is net salary calculated? Gross salary minus the unpaid-leave deduction.
10. Why is vector used? It stores a flexible number of Employee objects during program execution.

APPENDIX A: HOW TO COMPILE AND RUN

Using a terminal with g++

28. Save Employee_Salary_Management_System.cpp in a folder.
29. Open Command Prompt or Terminal in that folder.
30. Compile the source using the command below.
31. Run the generated program and follow the menu.

```
g++ -std=c++17 Employee_Salary_Management_System.cpp -o salary_system
./salary_system  (Linux/macOS)
salary_system.exe  (Windows)
```

Using an IDE

32. Open your C++ IDE.
33. Create a new C++ source file or open the provided .cpp file.
34. Paste or load the complete source code.
35. Build/Compile the project.
36. Run the program and enter the requested employee details.

If your IDE uses an older compiler, select a C++11 or later language standard because the program uses vector::emplace_back and range-based for loops.

APPENDIX B: GLOSSARY

Term	Meaning
OOP	Object-Oriented Programming.
Class	Blueprint that defines data and behaviour.
Object	Instance of a class.
Encapsulation	Bundling data and methods with controlled access.
Constructor	Special member function used to initialize an object.
Destructor	Special member function called when an object is destroyed.
TA	Travel Allowance.
DA	Dearness Allowance.
HRA	House Rent Allowance.
Gross Salary	Total earnings before the deductions considered here.
Net Salary	Amount after subtracting the stated deduction.
Vector	A dynamic sequence container from the C++ Standard Library.

APPENDIX C: PROJECT REVIEW

Project title: Employee Salary Management System

Student: Aditya

UID: 25BCA10391

Course: BCA – 3rd Semester

Programming language: C++

Core concept: Object-Oriented Programming

Project type: Console-based academic application

Faculty remarks

_____

_____

_____

_____

_____

Marks / Grade: _____

Faculty Signature: _____

Date: _____